Lab 3.2 – Queue Producer and Consumer with C#

Module M8 · Estimated time: 45–60 minutes

Scenario

An order-processing application sends messages to Azure Queue Storage. A consumer receives messages, but a process can crash after receiving a message and before deleting it. You must observe what happens when the message becomes visible again.

Objective

- Send five order messages.
- Peek without changing message visibility.
- Receive and delete a message.
- Simulate a crash before deleting another message.
- Observe redelivery and `DequeueCount`.
- Extend message visibility for a slow-running operation.

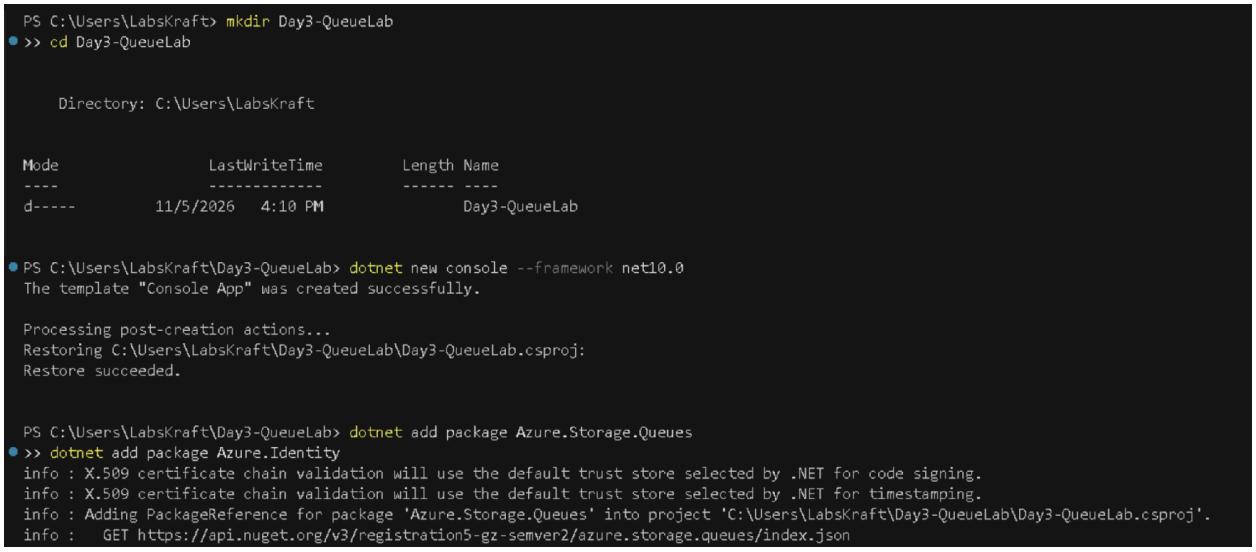
Step 1 – Create the C# project

Open VS Code terminal:

```
mkdir Day3-QueueLab
cd Day3-QueueLab

dotnet add package Azure.Storage.Queues
dotnet new console --framework net10.0
```

```
dotnet add package Azure.Identity
```

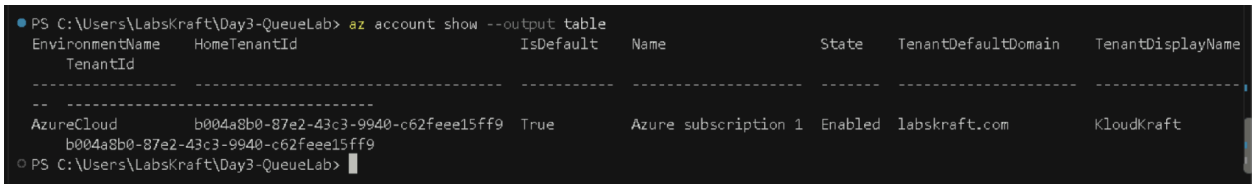


Step 2 – Configure Queue access

Use the Day 3 Storage Account and queue created during participant setup.

The signed-in identity must have Storage Queue Data Contributor on the Storage Account.

```
az login
az account show --output table
```



Step 3 – Create `Program.cs`

Replace the complete contents of `Program.cs`:

```
using Azure.Identity;
```

```
using Azure.Storage.Queues;
using Azure.Storage.Queues.Models;
using System.Text.Json;

const string storageAccount = "stuser01ordersd3";
const string queueName = "orders";

var queueUri = new Uri(

$"https://{storageAccount}.queue.core.windows.net/{queueNam
e}");

var credential = new DefaultAzureCredential();

var queue = new QueueClient(queueUri, credential);

await queue.CreateIfNotExistsAsync();

Console.WriteLine("Queue ready.");

if (args.Length == 0)
{
    Console.WriteLine(
        "Usage: dotnet run -- send | peek | receive | crash
| redelivery | visibility");
    return;
}

switch (args[0].ToLowerInvariant())
{
    case "send":
        await SendOrders();
```

```
        break;

    case "peek":
        await PeekOrders();
        break;

    case "receive":
        await ReceiveAndDelete();
        break;

    case "crash":
        await SimulateCrash();
        break;

    case "redelivery":
        await CheckRedelivery();
        break;

    case "visibility":
        await ExtendVisibility();
        break;

    default:
        Console.WriteLine("Unknown command.");
        break;
}

async Task SendOrders()
{
    for (int i = 1; i <= 5; i++)
    {
        var order = new
```

```
        {
            orderId = $"D3-ORD-{i:000}",
            sku = $"SKU-{100 + i}",
            qty = i
        };

        string json = JsonSerializer.Serialize(order);

        if (i == 5)
        {
            await queue.SendMessageAsync(
                json,
                timeToLive: TimeSpan.FromSeconds(60));
        }
        else
        {
            await queue.SendMessageAsync(json);
        }

        Console.WriteLine($"Sent: {json}");
    }
}

async Task PeekOrders()
{
    var response = await
queue.PeekMessagesAsync(maxMessages: 5);

    foreach (var message in response.Value)
    {
        Console.WriteLine(message.MessageText);
    }
```

```
}

async Task ReceiveAndDelete()
{
    var response = await queue.ReceiveMessagesAsync(
        maxMessages: 5,
        visibilityTimeout: TimeSpan.FromSeconds(30));

    foreach (var message in response.Value)
    {
        Console.WriteLine($"Received:
{message.MessageText}");
        Console.WriteLine($"Dequeue count:
{message.DequeueCount}");

        await queue.DeleteMessageAsync(
            message.MessageId,
            message.PopReceipt);

        Console.WriteLine("Message deleted.");
    }
}

async Task SimulateCrash()
{
    var response = await queue.ReceiveMessagesAsync(
        maxMessages: 2,
        visibilityTimeout: TimeSpan.FromSeconds(30));

    var messages = response.Value;

    if (messages.Length < 2)
```

```
    {
        Console.WriteLine("At least two visible messages
are required.");
        Console.WriteLine("Run the send command first.");
        return;
    }

    await queue.DeleteMessageAsync(
        messages[0].MessageId,
        messages[0].PopReceipt);

    Console.WriteLine($"Deleted first message:
{messages[0].MessageText}");

    Console.WriteLine(
        $"Simulated crash before deleting second message:
{messages[1].MessageText}");
}

async Task CheckRedelivery()
{
    Console.WriteLine("Waiting 32 seconds for message
visibility to expire...");
    await Task.Delay(TimeSpan.FromSeconds(32));

    var response = await queue.ReceiveMessagesAsync(
        maxMessages: 5,
        visibilityTimeout: TimeSpan.FromSeconds(30));

    foreach (var message in response.Value)
    {
        Console.WriteLine($"Redelivered:
```

```
{message.MessageText}");
        Console.WriteLine($"Dequeue count:
{message.DequeueCount}");

        await queue.DeleteMessageAsync(
            message.MessageId,
            message.PopReceipt);
    }
}

async Task ExtendVisibility()
{
    var response = await queue.ReceiveMessagesAsync(
        maxMessages: 1,
        visibilityTimeout: TimeSpan.FromSeconds(10));

    var message = response.Value.FirstOrDefault();

    if (message == null)
    {
        Console.WriteLine("No visible message found. Run
send first.");
        return;
    }

    Console.WriteLine($"Received: {message.MessageText}");

    await queue.UpdateMessageAsync(
        message.MessageId,
        message.PopReceipt,
        visibilityTimeout: TimeSpan.FromSeconds(60));
```

```
    Console.WriteLine("Visibility extended by 60
seconds.");
}
```

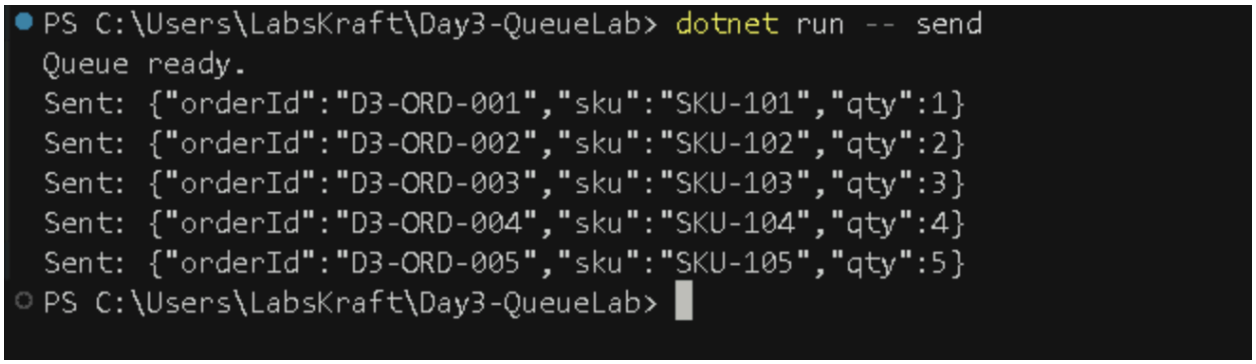
Important: Change `stuser01ordersd3` if you used a different globally unique Storage Account name.

Step 4 – Send five messages

```
dotnet run -- send
```

Expected:

- Five messages sent.
- The fifth message has a 60-second TTL.
- The other four use the default TTL.



Step 5 – Peek at messages

```
dotnet run -- peek
```

Expected: messages are displayed, but they remain available for other consumers. Peek does not hide messages.

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- peek
Queue ready.
{"orderId":"D3-ORD-001","sku":"SKU-101","qty":1}
{"orderId":"D3-ORD-002","sku":"SKU-102","qty":2}
{"orderId":"D3-ORD-003","sku":"SKU-103","qty":3}
{"orderId":"D3-ORD-004","sku":"SKU-104","qty":4}
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Step 6 – Receive and delete messages

```
dotnet run -- receive
```

Expected:

- Visible messages are received.
- Their dequeue count is displayed.
- The received messages are deleted using their message IDs and pop receipts.

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- receive
Queue ready.
Received: {"orderId":"D3-ORD-001","sku":"SKU-101","qty":1}
Dequeue count: 1
Message deleted.
Received: {"orderId":"D3-ORD-002","sku":"SKU-102","qty":2}
Dequeue count: 1
Message deleted.
Received: {"orderId":"D3-ORD-003","sku":"SKU-103","qty":3}
Dequeue count: 1
Message deleted.
Received: {"orderId":"D3-ORD-004","sku":"SKU-104","qty":4}
Dequeue count: 1
Message deleted.
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Step 7 – Simulate a consumer crash

First send a fresh batch:

```
dotnet run -- send
```

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- send
Queue ready.
Sent: {"orderId":"D3-ORD-001","sku":"SKU-101","qty":1}
Sent: {"orderId":"D3-ORD-002","sku":"SKU-102","qty":2}
Sent: {"orderId":"D3-ORD-003","sku":"SKU-103","qty":3}
Sent: {"orderId":"D3-ORD-004","sku":"SKU-104","qty":4}
Sent: {"orderId":"D3-ORD-005","sku":"SKU-105","qty":5}
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Now simulate the crash:

```
dotnet run -- crash
```

Expected:

- First received message is deleted.
- Second message is deliberately left undeleted.
- Its visibility timeout is 30 seconds.

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- crash
Queue ready.
Deleted first message: {"orderId":"D3-ORD-001","sku":"SKU-101","qty":1}
Simulated crash before deleting second message: {"orderId":"D3-ORD-002","sku":"SKU-102","qty":2}
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Step 8 – Observe redelivery

```
dotnet run -- redelivery
```

Expected: the undeleted message becomes visible again after the visibility timeout. Its dequeue count should be 2.

The first message should not reappear because it was deleted.

The message with a 60-second TTL may expire if you wait too long. Complete this part promptly.

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- redelivery
Queue ready.
Waiting 32 seconds for message visibility to expire...
Redelivered: {"orderId":"D3-ORD-003","sku":"SKU-103","qty":3}
Dequeue count: 1
Redelivered: {"orderId":"D3-ORD-004","sku":"SKU-104","qty":4}
Dequeue count: 1
Redelivered: {"orderId":"D3-ORD-002","sku":"SKU-102","qty":2}
Dequeue count: 2
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Step 9 – Extend visibility

Send another batch:

```
dotnet run -- send
```

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- send
Queue ready.
Sent: {"orderId":"D3-ORD-001","sku":"SKU-101","qty":1}
Sent: {"orderId":"D3-ORD-002","sku":"SKU-102","qty":2}
Sent: {"orderId":"D3-ORD-003","sku":"SKU-103","qty":3}
Sent: {"orderId":"D3-ORD-004","sku":"SKU-104","qty":4}
Sent: {"orderId":"D3-ORD-005","sku":"SKU-105","qty":5}
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Then:

```
dotnet run -- visibility
```

Expected: one message is received and its visibility is extended by 60 seconds.

```
PS C:\Users\LabsKraft\Day3-QueueLab> dotnet run -- visibility
Queue ready.
Received: {"orderId":"D3-ORD-001","sku":"SKU-101","qty":1}
Visibility extended by 60 seconds.
PS C:\Users\LabsKraft\Day3-QueueLab>
```

Acceptance criteria

Test	Expected result
Producer	Five messages sent
Peek	Messages visible without being hidden
Delete	Deleted message does not reappear
Simulated crash	Undeleted message becomes visible again
Redelivery	Dequeue count becomes 2
Visibility update	Message remains invisible for extended duration

Production takeaway: Queue Storage provides at-least-once delivery, not exactly-once processing. A consumer must be designed to handle duplicate deliveries safely. For ordering, sessions, topics and built-in dead-letter capabilities, evaluate Azure Service Bus.